Linux troubleshooting – the commands and words admins really use

One page to keep open when a ticket says "the server is slow / froze / an app stopped" · from Project 2 (tickets #5–#9)

1 · The first 5 minutes – always the same order

#	Question	Command	Look for
1	Did it restart? How busy is it?	`uptime` · `last reboot | head -3` · `journalctl --list-boots | tail -3`	short uptime = restart · load higher than CPU count = busy
2	Is a service broken?	`systemctl --failed`	any red "failed" unit
3	Any errors in this / the last run?	`journalctl -p err -b` · `journalctl -p err -b -1`	errors near the time of the ticket
4	Did the kernel see trouble?	`journalctl -k -p warning -b` (or the master filter, part 3)	the words in part 2
5	CPU, RAM or disk – right now?	`vmstat 1 5` · `free -h` · `top`	**r** high = CPU · **si/so** > 0 = RAM short · **b, wa** high = waiting for disk
6	Is a disk full?	`df -h` · `df -i` · `du -sh /var/* | sort -h | tail`	Use% 90–100 % · inodes 100 % · deleted-but-open: `lsof +L1`
7	Is the disk slow?	`iostat -xz 1 3` (package sysstat) · `ps -eo pid,stat,cmd | awk '$2 ~ /D/'`	%util near 100, high await · programs in state **D**
8	Does the network / the port answer?	`ip -br a` · `ip route` · `ping -c 3 <host>` · `ss -tlnp` · `nc -zv <host> <port>`	DOWN · no default route · "Connection refused" (nothing listens) · "timed out" (blocked)
9	What did one app say?	`journalctl -u <service> --since "1 hour ago"` · `tail -n 50 /var/log/<app>/error.log`	the app's own error at the ticket's time

Time window for every journalctl: `--since "2026-11-07 19:15" --until "2026-11-07 19:30"` · `-b -1` = the run before (check `--list-boots` first – the numbers move after every restart).

2 · The kernel words admins search for – and what they mean

Word in the kernel log	What happened	Typical ticket	First fix
`Out of memory · oom-kill · Killed process`	RAM and swap full – the kernel killed a program	"the app suddenly stopped"	which process was big (`ps aux --sort=-%mem | head`), add RAM, set MemoryMax
`soft lockup · CPU stuck · rcu … stall`	a CPU got no time for seconds/minutes	"the server froze, then came back" (**our ticket #9**)	check the host / hypervisor, overload, firmware
`hung_task · blocked for more than 120 seconds`	a program waited for the disk (state D) too long	"everything hangs", storage problem	check the disk / NFS / SAN, `iostat`
`I/O error · Buffer I/O error · blk_update_request`	the disk could not read/write a block	"files are broken", slow, crashes	disk dying – backup now, replace disk, `smartctl`
`EXT4-fs error · Remounting filesystem read-only`	the file system found damage and protected itself	"cannot save anything"	fix with `fsck` (unmounted), check the disk
`Link is Down · NETDEV WATCHDOG · Reset adapter · timed out`	the network card lost its link or hung	"server not reachable for a while"	cable/switch/virtual switch, driver, host
`segfault`	a program crashed (bad memory access)	"the app crashed"	app log, update/bug report
`nf_conntrack: table full, dropping packet`	the firewall's connection table is full	"random connection failures"	raise `nf_conntrack_max`, find the flood
`temperature above threshold · throttled`	CPU too hot – slowed down	"slow only sometimes" (real hardware)	fans, dust, air-con
`Call Trace · BUG: · Oops`	a kernel bug or driver problem (the lines after it say where)	freezes, crashes	update kernel/driver; report

3 · One master filter – run it first

```
journalctl -k -b -p warning --no-pager | grep -i -E "out of memory|oom|killed process|soft lockup|stuck|hung_
task|blocked for more than|i/o error|ext4-fs error|read-only|link is down|reset adapter|timed out|segfault|ca
ll trace|conntrack.*full|throttl"
```

`-k` kernel only · `-b` this run (use `-b -1` for the run before, or add `--since/--until`) · `-p warning` warnings and worse · `grep -i -E` = any of these words, upper or lower case (the dash before **E** matters).

Make it one word – `kcheck`: save as a small script and run `kcheck` (this run) or `kcheck -1` (the run before).

```
sudo tee /usr/local/bin/kcheck >/dev/null <<'EOF'
#!/bin/bash
# kcheck — first look at the kernel log. Usage: kcheck [boot]   e.g. kcheck -1
B=${1:-0}
echo "== runs =="; journalctl --list-boots | tail -3
echo "== kernel problems in run $B =="
journalctl -k -b "$B" -p warning --no-pager | grep -i -E "out of memory|oom|killed process|soft lockup|stu
ck|hung_task|blocked for more than|i/o error|ext4-fs error|read-only|link is down|reset adapter|timed out|
segfault|call trace|conntrack.*full|throttl" || echo "nothing found"
echo "== failed services =="; systemctl --failed --no-legend || true
EOF
sudo chmod +x /usr/local/bin/kcheck
kcheck -3
```

4 · Ticket → where to look (what we did in tickets #5–#9)

Ticket says	Check	Answer we found
"slow the first time, fast after"	`time` twice + `vmstat 1`: bi, cache	file not in RAM (#5)
"saved – is it really on the disk?"	`df` → `ls` → `grep Dirty /proc/meminfo`	Dirty 0 = on the disk (#6)
"saving is slow"	`ps` state D + `vmstat` b, wa	waiting for the disk, not CPU (#7)
"did the night job run?"	`journalctl -t <tag>`	the script's own log lines (#8)
"the shop froze 3–4 min"	app log → **kernel log** → DB log → network	soft lockup, CPU stuck 160 s (#9)

Final paragraph: Every "slow / froze / stopped" ticket starts the same way: did it restart (uptime, last reboot, list-boots), is a service failed (systemctl --failed), what errors are in this and the last run (journalctl -p err), what did the kernel see (the master filter or kcheck), and only then is it CPU, RAM, disk or network right now (vmstat, free, df, iostat, ss, ping). The kernel uses a small set of words for real trouble – out of memory and killed process for full RAM, soft lockup and CPU stuck for a frozen CPU, hung_task for long disk waits, I/O error and read-only for a failing disk or file system, link is down and reset adapter for the network, segfault for a crashed program, and call trace for kernel bugs. One grep with those words finds them all, and the kcheck script turns it into one command; in ticket #9 it shows "soft lockup – CPU stuck for 160s", "Watchdog timeout" and "Reset adapter" at 19:22, which proves the whole app01 froze.